YAFL

Yet Another f... Language

Fabian Becker

January 15, 2026

Abstract

This report details the design philosophy and implementation of the `YAFL` programming language. Submitted as the final study project for the Compiler Construction course, `YAFL` is a statically typed, general-purpose procedural language compiling to bytecode for a modified version of the provided `VM3` virtual machine.

1 General Overview

`YAFL` stands for "Yet another f... language". The "f..." can be replaced based on the programmer's current stance towards the language: *Fabulous* when it compiles, *Fun* when writing algorithms, or *Frustrating* when debugging.

1.1 Design Philosophy

The core idea behind `YAFL` is to merge the readability and expressiveness of modern languages with the structural discipline of traditional languages to provide readable programs with clear, predictable behavior at runtime. While the function signatures can be considered quite similar to Go, it also borrows high-level concepts like generalized `for` loops and `range` iterables from Python, making common tasks concise. However, it retains the explicit structure of C-like languages, using braces `{}` for blocks and semicolons `;` for statement termination which is familiar to almost every programmer as well as a static type system to catch issues at compile time and not at runtime.

1.2 Compiler Architecture

The YAFL compiler is implemented in C and utilizes a pipelined architecture to transform source code into executable bytecode. The compilation process consists of the following stages:

1. **Lexical Analysis:** Implemented using `Flex`, this stage scans the input source code and converts character streams into a sequence of tokens.
2. **Parsing:** Implemented using `Bison`, this stage consumes the token stream to construct the Abstract Syntax Tree (AST), representing the hierarchical structure of the program.
3. **Optimization:** An optional pre-processing pass traverses the AST to perform Constant Folding and Dead Code Elimination. This simplifies the tree structure before it is processed further.
4. **Code Generation & Semantic Analysis:** The AST is traversed to lower the program into the custom bytecode instruction set for the `VM3` virtual machine. During this traversal, semantic analysis is performed on-the-fly. This includes symbol resolution, strict type checking, and argument validation, ensuring that only valid and type-safe programs result in executable bytecode.

2 Syntax and Conventions

YAFL introduces several unique syntactic elements that distinguish it from standard C-family languages.

2.1 Identifiers and Variables

Variables are visually distinct, enclosed in pipe characters (e.g., `|my var|`). This design choice immediately separates variable references from keywords and function names. Moreover YAFL places almost no restrictions on these Identifiers, only pipes and newline characters are prohibited, allowing for more expressive variable names.

2.2 Numbers

Various way to express numbers are supported in YAFL, besides the standard of writing decimal numbers. You can prefix your number with a hashtag # to denote Hexadecimal numbers, moreover you can prefix any base from 2-16 before the hashtag and the number behind is interpreted in the base.

This also works for floating point numbers, here both the parts before and after the decimal point are interpreted in this base. Only the optional exponent expressed by the caret (^) needs to be a decimal.

In order to grasp large numbers better they can be formatted with the underscore, this formatter is ignored during parsing and works for both integers and floating point numbers.

```
    int |hex1| <- #FF; ~~ 255
    int |hex2| <- 16#A0; ~~ 160
    int |dozen| <- 12#A0; ~~ 10 * 12 + 0 = 120

    float |hex_fl| <- 16#F.8; ~~ 15 + 8/16 = 15.5
    float |exp_fl| <- 10#1.5^2; ~~ 1.5 * 10^2 = 150.0

    int |large number| <- 10#1_000_000;
```

2.3 Assignments

The assignment operator is <-, emphasizing the flow of data into the variable. Combined arithmetic operations (Addition, Subtraction, Multiplication, Division & Modulo) and assignments are also supported for variables. Simple increment and decrement operators are also implemented with the familiar `|my var|++` and `|my var|--`, however these can be only applied after the Identifiers and execute the operation and return the updated value.

```
int |count| <- 0;

~~ |count| <- |count| + 1 can be expressed as
|count| <-+ 1;
|count|++;
```

2.4 Strings

String literals are delimited by double chevrons (>>...<<). This avoids common escaping issues found with single or double quotes and allows for easy embedding of other quote characters. The caret (^n) serves as the escape characters and arbitrary characters can be inserted by their hexcode (^xAA).

```
str |msg| <- >>Hello, "World"!^n<<;
```

2.5 Comments

YAFL supports both line and block comments:

- **Line comments**: Start with `~~` and extend to the end of the line.
- **Block comments**: Enclosed within `{- ... -}`.

2.6 Function Signatures

Function declarations explicitly separate parameter types and names, and use `->` to indicate the return type.

```
fn calculate(int : |x|, int : |y|) -> int { ... }
```

In resemblance to assembly programs a valid YAFL program needs a function named `start()` with no parameters. The return type can be either `int` or `none`, however in both cases nothing is returned.

```
fn start() -> <int|none> { ... }
```

3 Language Features

3.1 Type System

YAFL is statically typed, ensuring type safety at compile time. It supports a variety of primitive and composite types.

3.2 Supported Types

- **Primitives**: `int` (signed integer), `float` (floating point with double precision), `bool` (`Yes`/`No`), and `str` (string).
- **Collections**: `arr` (typed arrays) and `range` (arithmetic sequences).
- **Special**: `none` (void) only used to indicate empty returns.

Primitive types are passed by reference, while Collections are passed by values. `arr` is a static array which allows for composite types like `arr'arr'str`.

3.3 Zero Initialization

Types are automatically set to their corresponding zero value if there is no assignment during the first definition. For `str` this is the empty string and for `bool` this is `No`. For arrays the corresponding primitive are used.

3.4 Input Arguments

Input arguments can be accessed by calling the `args()` function. It returns a string array with the input arguments, however it doesn't start with the program name but rather the first other argument.

3.5 Control Structures

3.5.1 If-elif-else Statements

If-statements are also implemented in the common way, were to condition needs to be of type `bool`. Sequential comparisons can be expressed via the `elif` keywords, but essentially these statements are converted to nested if-else during AST generation.

```
if |i| % 15 = 0 { ... }
elif |i| % 3 = 0 { ... }
else { ... }
```

3.5.2 While Statements

While loops are also implemented in a known manner, colons for the condition of type `bool` are not needed.

```
while Yes { ... }
```

3.5.3 For Loops and Iterables

The `for` loop is a generalized iterator. It abstracts over different data sources using the syntax `for <type> |var| in iterator`.

Ranges and Arrays: The built-in `range()` function generates arithmetic sequences. Uniquely, `YAFL` supports iteration over array literals and strings directly.

The `range()` is a special lazy iterator as it does not create the array on the stack which would result in huge memory consumption for large iterations. Rather the `range()` is represented as an array of following structure on the stack `["__RANGE__", <current>, <stop>, <step>]`. Current is initially set to the start value. The OPCODE `MKRANGE` creates this special array on stack from provided start, stop and step values, this syntax can't be recreated by user, as arrays need to have a common type. Ranges can only be of type `int`, and support negative start and stop values to begin iteration from the high

Arrays and strings also get wrapped in an iterator container of the structure [<object>, <current>, <len>] by the OPCODE `MKITER`.

The iteration process is then performed by the OPCODE `ITER_NEXT`, here either the next value of the iterator is pushed and a one is pushed to the stack and the container is updated or the container gets popped and a zero is pushed to the stack to indicate the end.

```
~~ Range with negative step (decimal notation)
for int |i| in range(10, 0, -2) { ... }

~~ Iterating over an array literal
for int |x| in [10, 20, 30] { ... }

~~ Iterating over a string
for str |x| in >>Yafl␣is␣fun!<< { ... }
```

3.5.4 Match Statements

The `match` statement replaces complex `if-else` chains. It supports all primitive types, so strings as well. The case expressions needs to be collapsible to literals at compile time so e.g. $1 + 5$ is allowed.

Since strings are also supported we cant simply build a jumptable in bytecode as it is not trivial to calculate the offset of a string. Liner scan on the other hand are essentially sequential if statements in bytecode which generate no performance improvement and therefore discards the idea of the match statement all together. Hence the Match statement was implemented using the map datatype of `MV3`, which is essentially a hashmap and used to e.g. store constants.

During code generation a map with the case expression as key and the program counter for the corresponding bytecode is generated. With the special OPCODE `SWITCH_LOOKUP` we pass this map, a default program counter location and the value to be compared, the operation then returns the correct location by performing a hashmap lookup.

```
match |val| {
    case 1 -> { print(>>One<<); }
    case 2 ->
    case 3 -> { print(>>Two or Three<<); }
    otherwise -> { print(>>Other<<); }
}
```

3.5.5 Next and Stop operators

Next and stop operator are the equivalent to break and continue. However these can only be used in for and while loops not the match statement as the break is handled here implicitly. During code generation a loop stack is used to keep track of the current of loop, if a loop is entered the information get pushed onto the stack. If a next or stop is occurred the stack is used to check if the current context contains a loop or not, moreover for next we can the access the jump target to move back to the loop start, if a stop is detected we push the program counter of the location where the jump target needs to be inserted onto the stack. After the loop is generated we patch these counters before popping the loop of the stack.

3.6 Functions

This language includes advanced function features, namely: Forward References, Overloading and Default Arguments

In order to support these features a custom symbol table was created. To use this table during code generation the OPCODE `CALL_PC` was created, which allows calling a function passed on a program counter rather than it's not unique function name.

3.6.1 Forward References

The code generation of `YAFL` uses a two pass approach. First all functions are registered in the custom symbol table, then in the second pass the function bodies are created and their program counters are updated in the table. If during the second pass a call to a function that is not yet generated occurs the corresponding location is appended to the called functions fixup list. When this function is then generated the fixup list is evaluated and the program counter locations are patched. If a call to an already generated function body occurs the location can be accessed via a symbol table lookup.

This way the function order doesnt matter and the programmer can group functions to be the most readable to them.

3.6.2 Overloading and Default Arguments

`YAFL` supports function overloading, allowing multiple definitions for the same name with different signatures. The signature includes the function name, amount of arguments, the type of the arguments and the order of the arguments.

It also supports default values for trailing arguments, simplifying API usage. When defining a default argument, all succeeding arguments need a default value as well. Default arguments are type checked as well during compilation.

```
fn log(str : |msg|, int : |level| <- 1) -> none { ... }
```

The `print` and `println` function use the internal type `any`, these can be overloaded for a more specific type, which for example was done for the `range` type.

3.6.3 Builtin Functions

A bunch of builtin functions are also supported these are essentially function pointer that point to a function with generated the bytecode for this function. They are registered at the symbol table during

before code generation and treated as function calls. This keep the lexer and parser more simple. A list of the functions can be found in the appendix.

4 Optimizations

During after code generation the compiler applies Constant Folding and Dead Code Elimination. For Dead Code Elimination we can collapse if-elif-else and match statements into just the correct statement if the comparison can be evaluated at compile time. Also all code after next and stop and ret (return) is considered unreachable and will be removed.

5 Toolchain

`YAFL` comes with a two-step toolchain similar to Java.

First `yaflc` is used to compile *.yafl* files to bytecode files *.yalfb* through command line arguments the compilation process can be observed at different levels of detail. The `-v` flag additionally generates an AST visualization both as a dot and svg file.

Then `yafl` is used to execute the *.yaflb* files on `VM3` you can also pass input argument after the bytecode file name into the program that can be accessed via the `args()` function.

A Project Structure

The submitted project is structured in the following way:

- **demo:** Contains the demonstration programs
- **doc:** The LaTeX code for this report
- **src:** Compiler source code
- **test:** Various small programs written during implementation
- **vm3:** The code for the modified version of `VM3`

B List of Builtin Functions

Signature	Description
`args() -> arr'str`	Returns the command-line arguments passed to the program.
`contains(str: |haystack|, str: |needle|) -> int`	Returns the starting index of `needle` in `haystack`, or -1 if not found.
`copy(<arr|str|range>: |val|) -> <type>`	Creates a deep copy of an array, string, or range.
`exit() -> none`	Immediately terminates the program.
`input_int(str: |prompt|) -> int`	Prints a prompt and reads an integer from stdin.
`input_str(str: |prompt|) -> str`	Prints a prompt and reads a line from stdin.
`len(<arr|str>: |val|) -> int`	Returns the number of elements in an array or characters in a string.
`print(any: |val|) -> none`	Prints the string representation of a value to stdout.
`println(any: |val|) -> none`	Prints the string representation of a value followed by a newline.
`range(int: |start|, int: |stop|, int: |step| <- 1) -> range`	Creates a range iterator from `start` to `stop` with step `step`.
`range(int: |stop|) -> range`	Creates a range iterator from 0 to `stop` with step 1.
`read(str: |path|) -> arr'str`	Reads a text file into an array of strings (lines).
`rng(int: |min|, int: |max|) -> int`	Generates a random integer between `min` and `max` (inclusive).
`rng(int: |max|) -> int`	Generates a random integer between 0 and `max` (inclusive).
`sleep(int: |ms|) -> none`	Pauses program execution for `ms` milliseconds.
`slice(<arr|str>: |v|, int: |s| <- 0, int: |e| <- MAX) -> <type>`	Returns a slice from index `s` (inclusive) to `e` (exclusive).
`to_bool(<int|float|str>: |v|) -> bool`	Converts or parses a value to a boolean.
`to_float(<int|bool|str>: |v|) -> float`	Converts or parses a value to a float.
`to_int(<float|bool|str>: |v|) -> int`	Converts or parses a value to an integer.
`to_str(<int|float|bool>: |v|) -> str`	Converts a value to its string representation.
`write(str: |p|, str: |txt|, bool: |app| <- No) -> bool`	Writes or appends text to a file. Returns success status.

C VM3 Modifications

To support `YAFL`'s features, the standard VM3 architecture was extended with custom opcodes and native functions. Besides the already mentioned ones, these are the additionally created opcodes.

OPCODE/NATIVE Identifier	Description
ARGS	Pushes commandline argument array to stack
CONTAINS	Returns the index of the needle (str) in the haystack (str), -1 if not found
READ_FILE	Reads a file by returning a string array of lines (newline gets stripped)
SLEEPMS	Sleeps for the provided number of milliseconds ($\leq 10s$) by using `nanosleep()`
SLICE	Slices a string, negative indexing (start from end) allowed, returns a deep copy
SWAP	Swaps the two topmost elements on the stack
WRITE_FILE	Writes or appends string to a file (without newline), returns success a boolean

Other other Modifications of the source code include:

- Renamed `stack_t` to `vmstack_t` due to name collision with stack.h required by signal.h.
- Rewrote type casting to include parsing of `YAFL`s custom number format from string. Added additional functionality for some types.
- Changed `exec_t` and `exec_new` to accept commandline arguments.
- `arr_copy` now creates deep copy even for nested types.
- `str_add_buf` uses exponential growth in order to trigger less reallocation when concatenating multiple strings.

D Demonstration Programs

The capabilities of `YAFL` are demonstrated through four included programs:

1. **Game of Life**: Implements Conway's Game of Life to showcase the language's handling of multi-dimensional arrays and nested loops. It utilizes the `sleep` builtin and ANSI escape codes to create an animated visualization in the terminal.
2. **Run Length Decoder**: A recursive descent parser for decoding run-length encoded strings (e.g., `3[ab]`). This program demonstrates recursion, string manipulation functions (`contains`, `slice`), and conditional logic.
3. **Black Jack**: A fully interactive card game simulation. It highlights the use of the `rng` builtin for game logic, `input_str` for user interaction, and extensive control flow using `match` and `if-else` structures.
4. **Language Demo**: A comprehensive showcase of specific language features including function overloading, slicing, and pattern matching.